

Experiment: 5

Aim: To containerize an application using a Docker container.

1 What is Containerization?

Containerization is the process of packaging an application together with its required dependencies and runtime environment into a container. This allows the application to run consistently on different systems.

Java Application + Java Runtime + Dependencies



Docker Image



Docker Container

2 What is a CLI Java Application?

A CLI (Command Line Interface) Java application is a Java program that runs through a terminal or command prompt instead of using a graphical user interface.

For example:

Enter your name:

Yash

Hello Yash!

The application accepts input and displays output through the command line.

3 Why Containerize a Java Application?

Normally, running a Java application requires a suitable Java Runtime Environment (JRE/JDK) and other required dependencies on the system. Different systems may have different Java versions or configurations, which can cause compatibility problems.

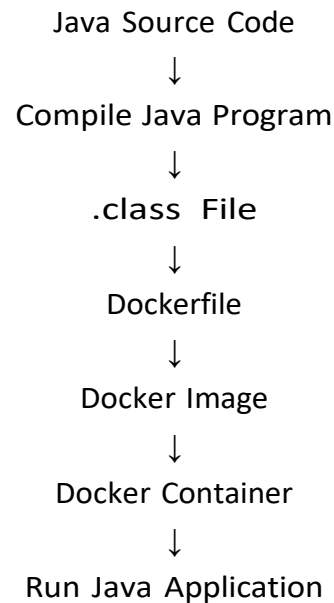
Docker addresses this by providing an isolated environment containing the required Java runtime and application.

Benefits

- Environment consistency
- Dependency management
- Portability
- Isolation
- Easy deployment
- Reproducibility

4 How is a Java Application Containerized?

The general process is:



5 Dockerfile in Java Containerization

A Dockerfile is a text file containing instructions for creating a Docker image. For a Java application, the Dockerfile generally specifies:

- Base Java image
- Application files
- Working directory
- Required configuration
- Command to execute the Java application

Dockerfile Build→ Java Docker Image Run→ Java Container

6 Docker Image for a Java Application

The Docker image contains everything required to run the Java application. For example, a Java Docker image contains:

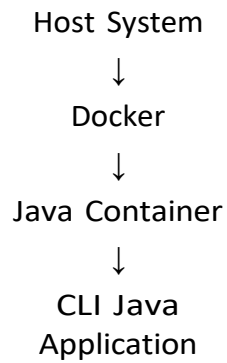
- Java runtime
- Application files
- Required configuration

The image acts as a blueprint for creating containers.

7 Docker Container for a Java Application

A Docker container is a running instance of the Java image. When the container starts, the Java application runs inside the container, and its CLI input/output can be accessed through an attached

terminal when run interactively.

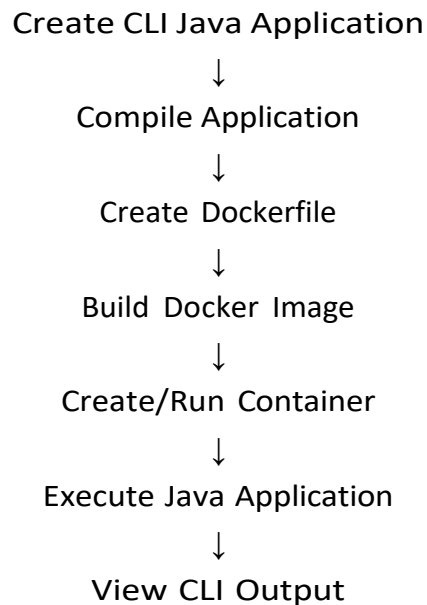


8 Important Difference

Term	Meaning
Java Application	Actual program
Dockerfile	Instructions for building an image
Docker Image	Blueprint containing the application environment
Docker Container	Running instance of an image

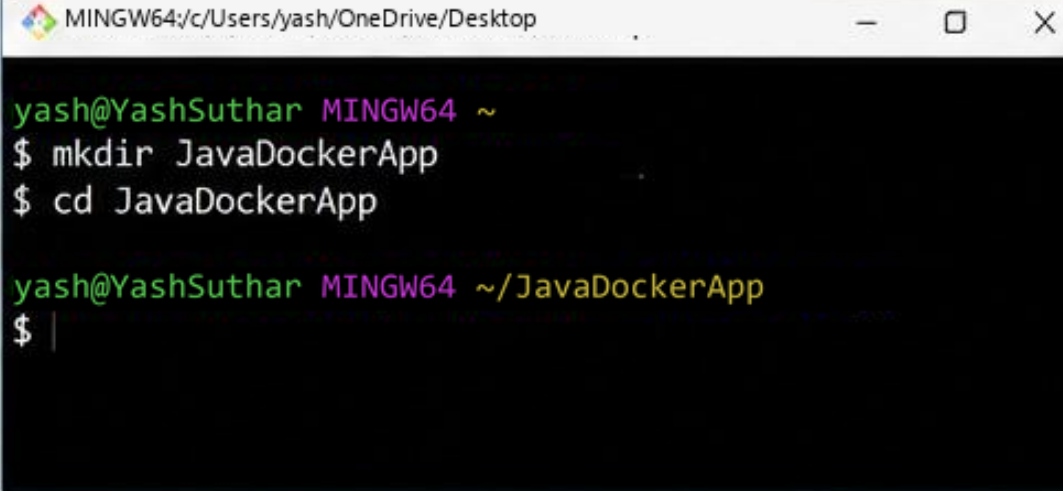
9 Overall Concept

For this practical, we will follow:



10 Implementation Screenshots

1. Create Project Directory



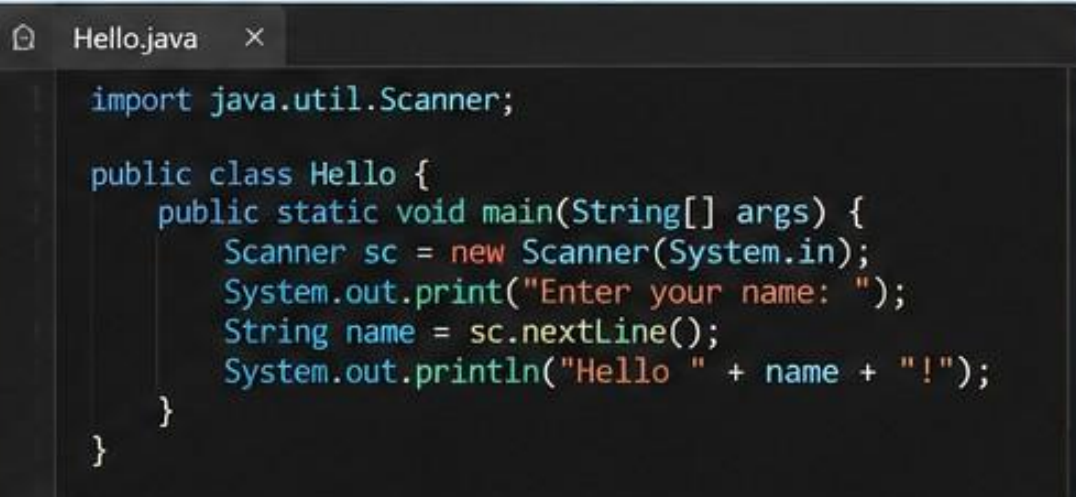
```
MINGW64:/c:/Users/yash/OneDrive/Desktop
yash@YashSuthar MINGW64 ~
$ mkdir JavaDockerApp
$ cd JavaDockerApp

yash@YashSuthar MINGW64 ~/JavaDockerApp
$ |
```

The screenshot shows a terminal window with a light blue title bar that reads "1. Create Project Directory". The window's address bar shows the path "MINGW64:/c:/Users/yash/OneDrive/Desktop". The terminal text shows the user "yash@YashSuthar" in a "MINGW64" environment at the "~" directory. They execute the command "\$ mkdir JavaDockerApp", followed by "\$ cd JavaDockerApp". The prompt then changes to "yash@YashSuthar MINGW64 ~/JavaDockerApp", and the user enters a dollar sign "\$" followed by a vertical bar "|", indicating the command prompt is ready for the next input.

Step 1: Create the JavaDockerApp project directory and navigate into it.

2. Java Source Code (Hello.java)



```
Hello.java x
import java.util.Scanner;

public class Hello {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = sc.nextLine();
        System.out.println("Hello " + name + "!");
    }
}
```

The screenshot shows a code editor window with a dark theme. The title bar of the editor shows "Hello.java" and a close button "x". The code is written in Java and includes an import statement for "java.util.Scanner". It defines a "public class Hello" with a "public static void main" method. Inside the main method, a "Scanner" object is created from "System.in", a prompt is printed, the user's input is read into a "String" variable named "name", and finally, a personalized greeting is printed using string concatenation. The code is properly formatted with indentation and closing braces.

Step 2: Write Hello.java to read a name and display a personalized greeting.

3. Dockerfile

```
Dockerfile x
# Use official OpenJDK image
FROM openjdk:17

# Set working directory
WORKDIR /app

# Copy compiled Java file
COPY Hello.class .

# Run the Java application
CMD ["java", "Hello"]
```

Step 3: Define a Dockerfile using OpenJDK 17, copy Hello.class, and set the Java startup command.

4. Compile the Java Application

```
MINGW64/c/Users/yash/OneDrive/Desktop/JavaDockerApp
yash@YashSuthar MINGW64 ~/JavaDockerApp
$ javac Hello.java

yash@YashSuthar MINGW64 ~/JavaDockerApp
$ ls
Hello.class  Hello.java  Dockerfile

yash@YashSuthar MINGW64 ~/JavaDockerApp
$
```

Step 4: Compile Hello.java with javac to generate Hello.class.

5. Build Docker Image

```
MINGW64/c/Users/yash/OneDrive/Desktop/JavaDockerApp
yash@YashSuthar MINGW64 ~/JavaDockerApp
$ docker build -t java-cli-app .
[+] Building 2.5s (6/6) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 149B
=> [internal] load metadata for docker.io/library/openjdk:17
=> [internal] load build context
=> => transferring context: 3.48kB
=> [1/2] FROM docker.io/library/openjdk:17
=> [2/2] COPY Hello.class .
=> exporting to image
=> => writing image sha256:1d3f5e6a2b8e6c9b7d...
=> => naming to docker.io/library/java-cli-app
yash@YashSuthar MINGW64 ~
```

Step 5: Build the java-cli-app image using `docker build -t java-cli-app .`

6. Verify Docker Image

```
MINGW64/c/Users/yash/OneDrive/Desktop/JavaDockerApp
yash@YashSuthar MINGW64 ~/JavaDockerApp
$ docker images

REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
java-cli-app        latest     1d3f5e6a2b8e  10 seconds ago 436MB
openjdk             17        6f3a2b4d9c1e  2 weeks ago   436MB

yash@YashSuthar MINGW64 ~/JavaDockerApp
$
```

Step 6: List local images with `docker images` to verify that java-cli-app was created.

7. Run the Container (Interact with CLI Application)

```
MINGW64:/c:/Users/yash/OneDrive/Desktop/JavaDockerApp
yash@YashSuthar MINGW64 ~/JavaDockerApp
$ docker run -it --name java-app java-cli-app
Enter your name: Yash
Hello Yash!

yash@YashSuthar MINGW64 ~/JavaDockerApp
$
```

Step 7: Run the container interactively, enter a name, and view the greeting.

8. List Running Containers

```
MINGW64:/c:/Users/yash/OneDrive/Desktop/JavaDockerApp
yash@YashSuthar MINGW64 ~/JavaDockerApp
$ docker ps
CONTAINER ID   IMAGE          COMMAND         CREATED        STATUS        PORTS          NAMES
2e4f8d9c1b7a   java-cli-app   "java Hello"    2 minutes ago  Up 2 minutes             java-app

yash@YashSuthar MINGW64 ~/JavaDockerApp
```

Step 8: Use `docker ps` to list containers that are currently running.

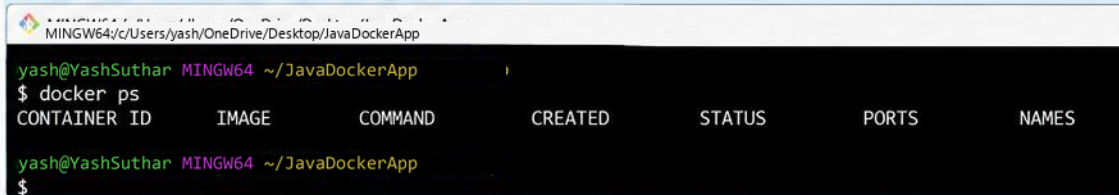
9. Stop and Remove Container



```
MINGW64:/c:/Users/yash/OneDrive/Desktop/JavaDockerApp
yash@YashSuthar MINGW64 ~/JavaDockerApp
$ docker stop java-app
java-app
yash@YashSuthar MINGW64 ~/JavaDockerApp
$ docker rm java-app
java-app
yash@YashSuthar MINGW64 ~/JavaDockerApp
```

Step 9: Stop and remove the java-app container using `docker stop` and `docker rm`.

10. Final Verification (No Running Containers)



```
MINGW64:/c:/Users/yash/OneDrive/Desktop/JavaDockerApp
yash@YashSuthar MINGW64 ~/JavaDockerApp
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

```
yash@YashSuthar MINGW64 ~/JavaDockerApp
$
```

Step 10: Run `docker ps` again to confirm that no containers are running.